

CE 488/5588: Applied AI
Topic 16: Modern LLM Systems and
Direct Engineering Applications —
Workflows, Retrieval, and First-Generation
AI Engineering

Lecture Roadmap

- 1 From Topic 15 to Topic 16
- 2 Mainstream Architectures of Modern LLM Systems
- 3 Base Models, Post-Training, and Assistant Behavior
- 4 Prompt Engineering as Workflow Design
- 5 First-Generation LLM Engineering Workflows
- 6 RAG and Knowledge Bases
- 7 Knowledge Graph Creation with LLMs
- 8 Direct LLM Applications in Engineering Work
- 9 Language as Programming, Language as Software

From Representation to Systems

Topic 15 focused on representation:

- how text becomes tokens,
- how tokens become vectors,
- how vectors support retrieval.

Topic 16 asks a different question:

- how are modern LLM systems actually organized,
- and how do those systems already change engineering work?

Why Topic 16 Matters

In practice, an engineering assistant is rarely just a raw pretrained model.

A usable system usually includes:

- prompting structure,
- post-training,
- retrieved evidence,
- external tools,
- human supervision.

Main Shift

Topic 16 is about the system stack around the model, not only the model itself.

What This Chapter Covers

This chapter focuses on direct LLM applications and first-generation LLM engineering.

It covers:

- mainstream transformer families,
- base models versus post-trained assistants,
- prompt engineering as workflow design,
- ReAct, RAG, and knowledge graphs,
- coding and document-centered engineering applications,
- the transition toward Topic 17 and more agentic systems.

Why LLM Systems Are More Than Chatbots

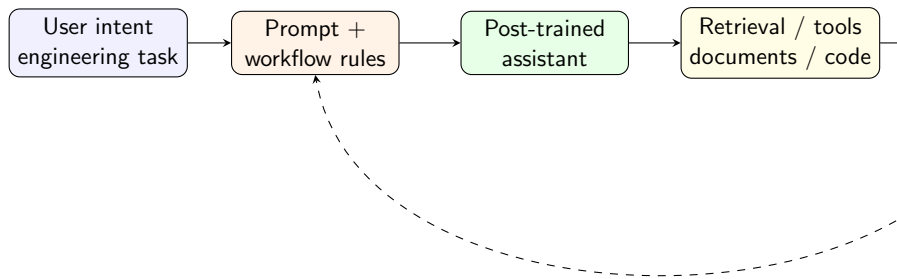


Figure: A practical system view of an engineering LLM assistant.

Interpreting the System Stack

The figure is useful because it makes three practical points explicit:

- prompts organize the interface between the user and the model,
- retrieval and tools bring in evidence the model does not reliably store,
- verification remains outside the model and inside human responsibility.

Useful AI engineering begins well before full autonomy.

Learning Objectives

After this chapter, students should be able to:

- distinguish encoder-only, decoder-only, and encoder-decoder families,
- explain base models versus assistants,
- describe prompt engineering as workflow design,
- explain RAG and knowledge-graph support for engineering knowledge systems,
- describe first-generation engineering workflows such as ReAct,
- explain why Topic 17 naturally follows from Topic 16.

Three Main Transformer Families

Although people often speak of “LLMs” as if they were one architecture, transformer-based systems come in three important families:

- encoder-only,
- decoder-only,
- encoder-decoder.

The engineering relevance is that these families are better suited to different task types.

Source: Hugging Face Course: Transformer Architectures

Encoder-Only Models

Encoder-only systems, such as BERT-style models, use bidirectional attention.

They are strong when the output is mainly an internal representation:

- classification,
- semantic similarity,
- retrieval,
- embedding generation.

A common objective is masked-token prediction:

$$\mathcal{L}_{\text{MLM}}(\theta) = - \sum_{t \in \mathcal{M}} \log P_{\theta}(x_t \mid x_{\setminus t}).$$

Decoder-Only Models

Decoder-only systems, such as GPT-style models, use causal attention.

This architecture naturally matches autoregressive generation:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t}).$$

This is why decoder-only models became the dominant front end for:

- chat,
- coding assistants,
- drafting,
- general-purpose generation.

Encoder-Decoder Models

Encoder-decoder systems, such as T5-style sequence-to-sequence models, split the pipeline into:

- an encoder for the source input,
- a decoder for the target sequence,
- cross-attention between them.

Their conditional objective has the form:

$$P(y \mid x) = \prod_{t=1}^{T_y} P(y_t \mid y_{<t}, x).$$

They are especially natural for translation, summarization, and structured text transformation.

Source: Hugging Face: Encoder Decoder Models

Architecture Comparison at a Glance

Architecture	Main strength	Typical use
Encoder-only	rich bidirectional representation	embeddings, classification, retrieval
Decoder-only	scalable prompt-driven generation	chat, coding, general assistants
Encoder-decoder	explicit input-to-output mapping	translation, summarization, structured generation

Three Mainstream Transformer Families

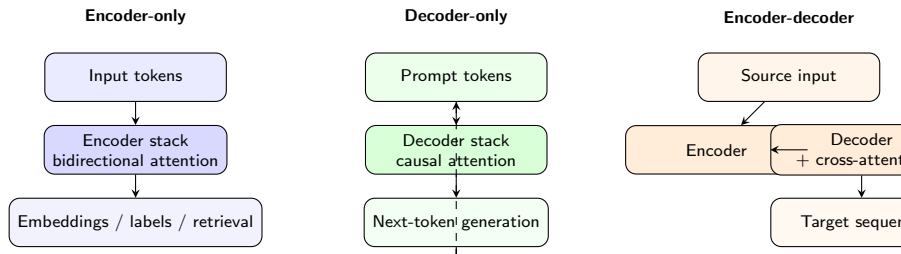


Figure: Three mainstream transformer families.

Why This Architecture Figure Matters

The figure is more useful here than a generic transformer diagram because Topic 16 is about system choices.

Design Consequence

Different architectural families support different engineering tasks for structural reasons, not just branding reasons.

If the task is embedding and search, encoder-style models are natural. If the task is interactive generation, decoder-only models dominate. If the task is explicit transformation from one text form to another, encoder-decoder models are conceptually clean.

Why Decoder-Only Models Dominated the Public Interface

After 2020, decoder-only systems drove the public AI wave because one architecture could support:

- chat,
- code completion,
- summarization,
- drafting,
- transformation,
- question answering.

This does not make the other families obsolete. It means decoder-only models became the most versatile public front end.

Base Model versus Assistant

A **base model** is the raw language model after pretraining.
A **post-trained assistant** is what results after additional behavioral shaping:

- instruction following,
- formatting reliability,
- tone and style preferences,
- refusal or safety behavior.

Practical Difference

What users experience is not just the architecture. It is the architecture plus post-training.

Instruction Tuning

Instruction tuning uses prompt-response demonstrations to teach task-following behavior.

If the training set is $\mathcal{D}$, the supervised objective is:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{(x,y) \in \mathcal{D}} \log P_{\theta}(y \mid x).$$

This is the step that helps convert “a model that can continue text” into “a model that behaves like an assistant.”

Preference Optimization

Not every desired behavior is a single correct answer.

Many goals are preference-like:

- helpfulness,
- professionalism,
- clarity,
- concision,
- tone.

RLHF and DPO are both attempts to shape outputs toward what people prefer, not only toward what is statistically likely.

Parameter-Efficient Customization

Organizations often want narrow customization without retraining every parameter.

That is why methods such as LoRA matter in practice:

- lower memory cost,
- lower engineering friction,
- easier adaptation to domain language,
- useful for internal styles, templates, or terminology.

Why This Matters for Engineers

For engineering organizations, many real use cases are narrow:

- report templates,
- compliance phrasing,
- project-specific terminology,
- domain-focused code support.

Those use cases often do not require frontier-scale retraining. They require system design around an already capable model.

Prompt Engineering Is Not Magic

Prompt engineering is sometimes presented as the search for one magical sentence.

That is not the most useful engineering view.

Better View

Prompt engineering is the design of a workflow interface between human intent and model behavior.

Prompt Components

A reliable prompt often specifies:

- role,
- context,
- task,
- constraints,
- verification cues.

The point is not mysticism. The prompt acts like a lightweight program written in natural language.

Source: LangChain Docs: Prompt Templates

A Minimal Prompt Template

Example Template

Role: You are a structural engineering assistant.

Context: Use only the attached design note and cited code clauses.

Task: Summarize the controlling assumptions for serviceability checks.

Constraints: Return 5 bullet points and cite the source section for each.

Verification: If a needed source is missing, say so explicitly.

This is already a workflow: it tells the model what to do, what evidence to use, and how to signal uncertainty.

Structured Prompting Patterns

Useful prompting patterns include:

- few-shot prompting,
- decomposition prompting,
- format-constrained prompting,
- verification prompting,
- tool-oriented prompting.

These matter because engineering tasks often require bounded outputs, traceable assumptions, and explicit uncertainty handling.

Prompt Engineering as Workflow Design

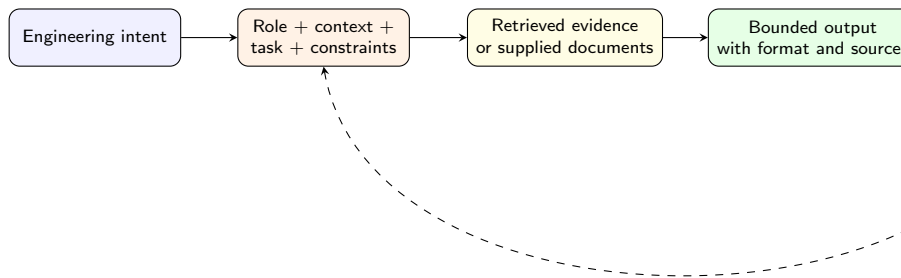


Figure: Prompt engineering as workflow design.

Why the Prompt-Workflow Figure Matters

The figure emphasizes that a strong prompt does more than ask for an answer.

It helps organize:

- what evidence matters,
- what format is required,
- how uncertainty should be expressed,
- what counts as success or failure.

Important Caution

A strong prompt can improve behavior, but it cannot create reliable evidence that is not present in the context.

Before Full Agents

Before frontier autonomous agents became a mainstream topic, many practical LLM workflows already existed. These are best understood as *first-generation AI engineering patterns*:

- more structured than ordinary chat,
- more tool-aware than plain prompting,
- but still short of full autonomy.

ReAct as a Bridge Pattern

ReAct stands for *reasoning and acting*.

The central idea is to interleave reasoning with action:

Thought $\rightarrow$ Action $\rightarrow$ Observation $\rightarrow$ Thought $\rightarrow \dots$

This makes the workflow iterative, tool-aware, and partially self-correcting.

Source: Yao et al. (2023): ReAct

A Simplified ReAct Loop

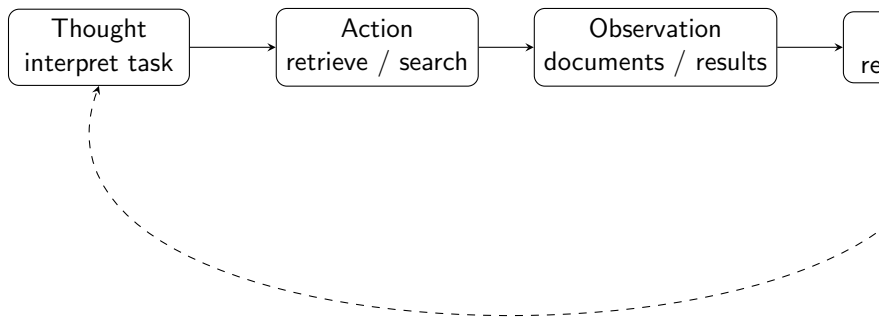


Figure: A simplified ReAct-style loop.

Why ReAct Matters in Engineering

Engineering tasks rarely succeed from one fluent answer.

The useful workflow is often:

- identify the missing source,
- retrieve the clause or document,
- inspect the evidence,
- revise the answer,
- document assumptions.

ReAct matters because it teaches the system to work with intermediate evidence instead of jumping directly to a final answer.

Engineering Example: Code Compliance

Serviceability Question

Suppose a user asks: “What serviceability deflection limit applies here?”

A ReAct-style workflow might:

- retrieve the relevant code section,
- observe that the clause depends on occupancy or finish sensitivity,
- request or infer that missing context,
- produce a bounded answer tied to evidence.

RAG as a System Pattern

Retrieval-augmented generation combines two ideas:

- 1 retrieve relevant external documents,
- 2 condition generation on those retrieved documents.

This shifts the core question from:

- “What does the model remember?”
- to “What evidence can the system retrieve and use?”

Source: Lewis et al. (2020): Retrieval-Augmented Generation

A Practical RAG Pipeline

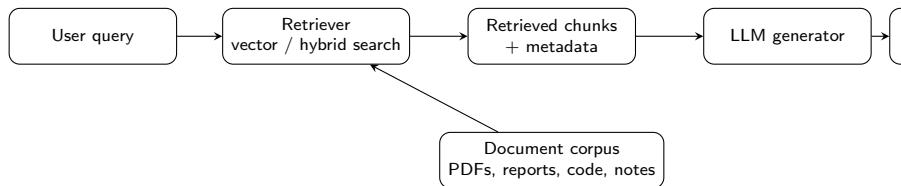


Figure: A practical RAG pipeline for engineering knowledge work.

Why the RAG Figure Matters

The figure highlights a crucial design principle:

- the retriever brings in evidence before generation,
- the answer depends on accessible context, not only model memory,
- metadata remains part of the practical retrieval story.

This is why RAG is especially natural for standards, reports, project archives, and office knowledge systems.

LangChain-Style Orchestration

Frameworks such as LangChain became popular because they made it easier to connect:

- prompt templates,
- retrievers,
- tools,
- memory,
- output formatting.

The important lesson is not one API. It is the application-layer orchestration pattern.

Source: LangChain Docs: Build a RAG Agent

Engineering Knowledge Bases

A useful engineering knowledge base is not just a pile of text. It is:

- indexed,
- chunked,
- traceable,
- version-aware,
- filtered by source type and authority.

The technical challenge is not only retrieval quality. It is also governance.

What Governance Means Here

For engineering work, a good knowledge system must help distinguish:

- official design codes,
- internal office standards,
- project-specific memos,
- advisory notes,
- superseded versus current sources.

Retrieval quality without source governance is not enough.

Why Knowledge Graphs Matter

RAG is not the only way to organize knowledge.
Knowledge graphs store entities and relations explicitly, often as triples:

$$(h, r, t)$$

Examples:

(ACI318, defines, load combination)

(Beam B12, uses material, A992 steel)

Text to Knowledge Graph Workflow

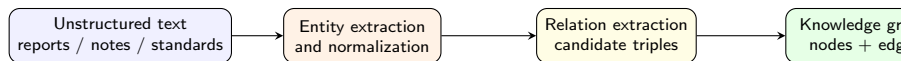


Figure: A simplified text-to-knowledge-graph workflow.

How LLMs Help Build Graphs

LLMs can assist with:

- entity extraction,
- relation extraction,
- alias resolution,
- ontology normalization,
- draft triple generation.

Important Limitation

A graph triple can look formally clean while still being semantically wrong. Human review remains necessary.

RAG and Knowledge Graphs Are Complementary

RAG is strong when you want grounded access to source passages.

Knowledge graphs are strong when you want:

- explicit relationships,
- structured queries,
- schema-aware reasoning,
- integration with rule-based logic.

Many useful engineering systems may combine both.

Document-Centered Applications

Before autonomy becomes the story, many direct applications already matter:

- summarizing long reports or standards,
- extracting assumptions, risks, and action items,
- comparing specification revisions,
- generating draft technical narratives,
- question answering over project archives.

These applications often create value by reducing knowledge bottlenecks rather than replacing final engineering authority.

Coding and Programming Assistance

LLMs are also becoming practical front ends for programming. Coding assistants can:

- explain unfamiliar codebases,
- draft scripts for analysis or visualization,
- translate intent into regex, SQL, or Python,
- propose tests, refactors, or documentation,
- help navigate multi-file repositories faster.

A Direct Coding-Assistant Workflow

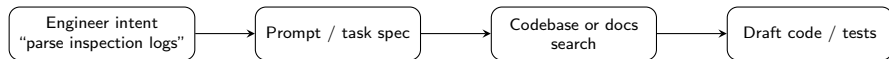


Figure: A direct coding-assistant workflow.

Why the Coding Workflow Figure Matters

The key point is that this is still a human-directed pipeline. Natural-language intent is translated into:

- search,
- draft generation,
- explicit verification.

It is more than autocomplete, but it is not yet unconstrained autonomy.

Examples of AI-Native Coding Front Ends

Tools such as GitHub Copilot, Claude Code, and OpenCode represent a broader interface shift.

The user increasingly describes an objective in language, and the system translates that into:

- repository search,
- code generation,
- edits,
- verification steps,
- planning support.

Engineering Example: Analysis Assistance

Sensor Drift Study

A student wants to compare sensor drift before and after recalibration.

Instead of starting from scratch, the assistant can:

- inspect the CSV schema,
- write a cleaning function,
- generate a plotting routine,
- explain assumptions in the analysis.

The useful output is not only code. It is faster movement from problem description to a runnable workflow.

A Broader Conceptual Shift

In classical software engineering, programming usually means writing formal code in languages such as Python, C++, or MATLAB.

In modern LLM workflows, natural language increasingly acts as a specification layer.

This does not replace formal code. It changes how objectives, constraints, and orchestration are expressed.

Language as Software Interface

Formal code still executes the work, but natural language increasingly specifies objectives, constraints, orchestration, and interaction patterns.

This matters in engineering because engineering work is already mediated by language: requirements, standards, reports, contracts, and code comments are all part of the computational ecosystem.

Why Topic 17 Must Go Further

Topic 16 leaves us with a natural next question:

- if prompts can orchestrate steps,
- if retrieval can ground answers,
- if ReAct can interleave thought and action,
- and if coding assistants can navigate tools,

what changes when those pieces become persistent, tool-using, and increasingly agentic?

Transition to Topic 17

What Comes Next

Topic 16 showed that useful AI engineering begins well before full autonomy. Topic 17 asks what changes when these first-generation workflows become persistent, tool-using, multi-step systems that look increasingly agentic.

That is where human-in-the-loop engineering becomes fully concrete.

Chapter Takeaways

- modern LLM systems are best understood as workflows, not just models,
- different transformer families support different task types,
- assistants feel different from base models because of post-training,
- prompt engineering is workflow design,
- ReAct, RAG, and knowledge graphs are first-generation engineering patterns,
- direct value often comes from document and coding workflows before autonomy.